



MapLibre Tile 3D Extension

Enabling Immersive Map Views

Markus Tremmel

Context

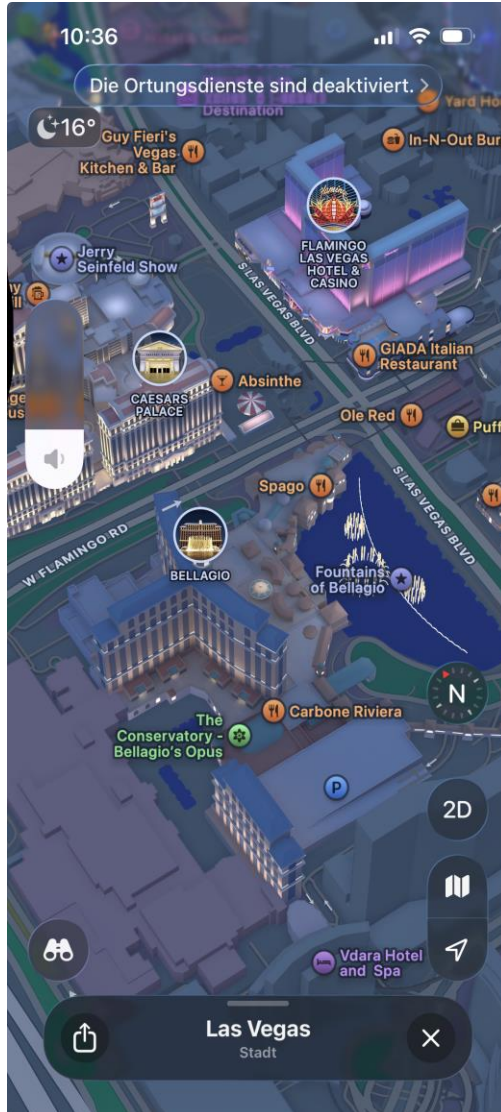
- The primary goal of the MLT 3D extension is to provide an efficient format for cartography-oriented 3D visualization, enabling low-latency rendering while preserving the familiar “slippy map” experience even on mobile devices and bandwidth-constrained networks
- It is based on the assumption that a moderate Level of Detail (e.g., LOD1–LOD2 for buildings) is sufficient for users to navigate and understand spatial relationships
- It is inspired by the immersive navigation view in Google Maps and the Detailed City Experience in Apple Maps
- It should enable the generation of immersive map views from open data sources such as OpenStreetMap and the Overture Maps Foundation



Source: Google Maps Platform Blog - <https://blog.google/products-and-platforms/products/maps/ask-maps-immersive-navigation/>

Goals: Cartography-Oriented 3D Visualization (Symbolised 3D Models)

Examples: Google Maps Immersive Navigation, Apple Maps Detailed City Experience

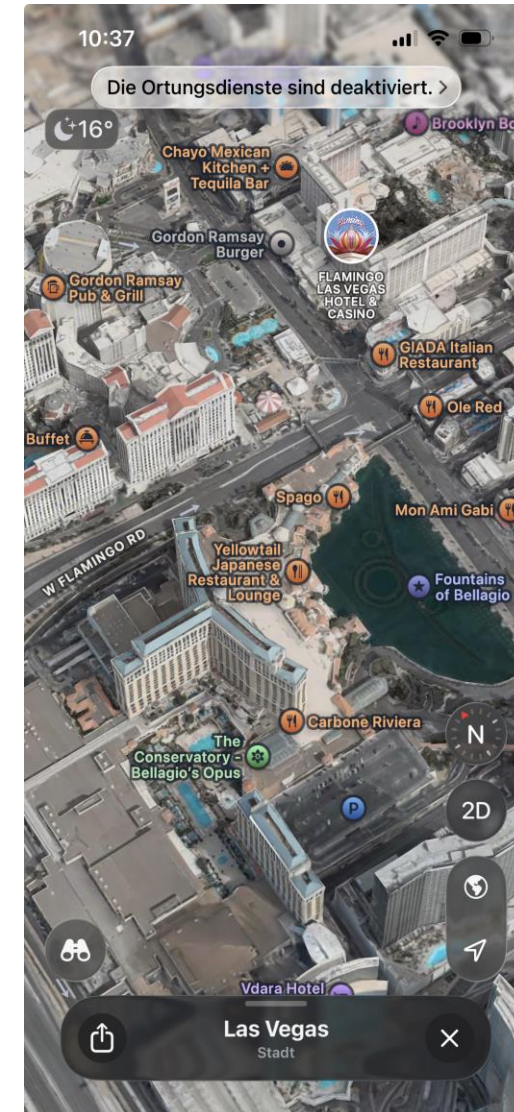


Source: Apple Maps screenshot © Apple Inc.

Non-Goals: Photorealistic Rendering

(not a primary focus, as this is already well supported by 3D Tiles)

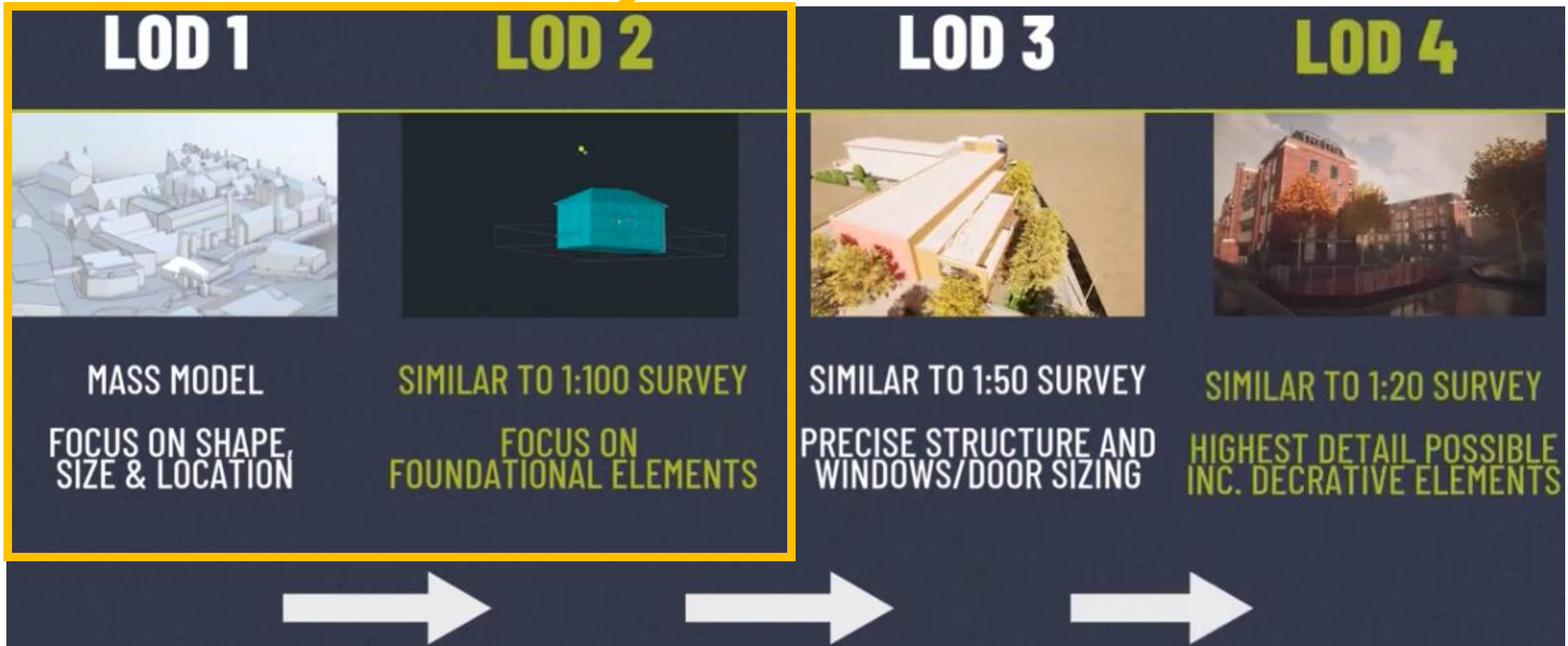
Examples: Google Maps Photorealistic 3D Tiles



Source: Apple Maps screenshot © Apple Inc.

LODs

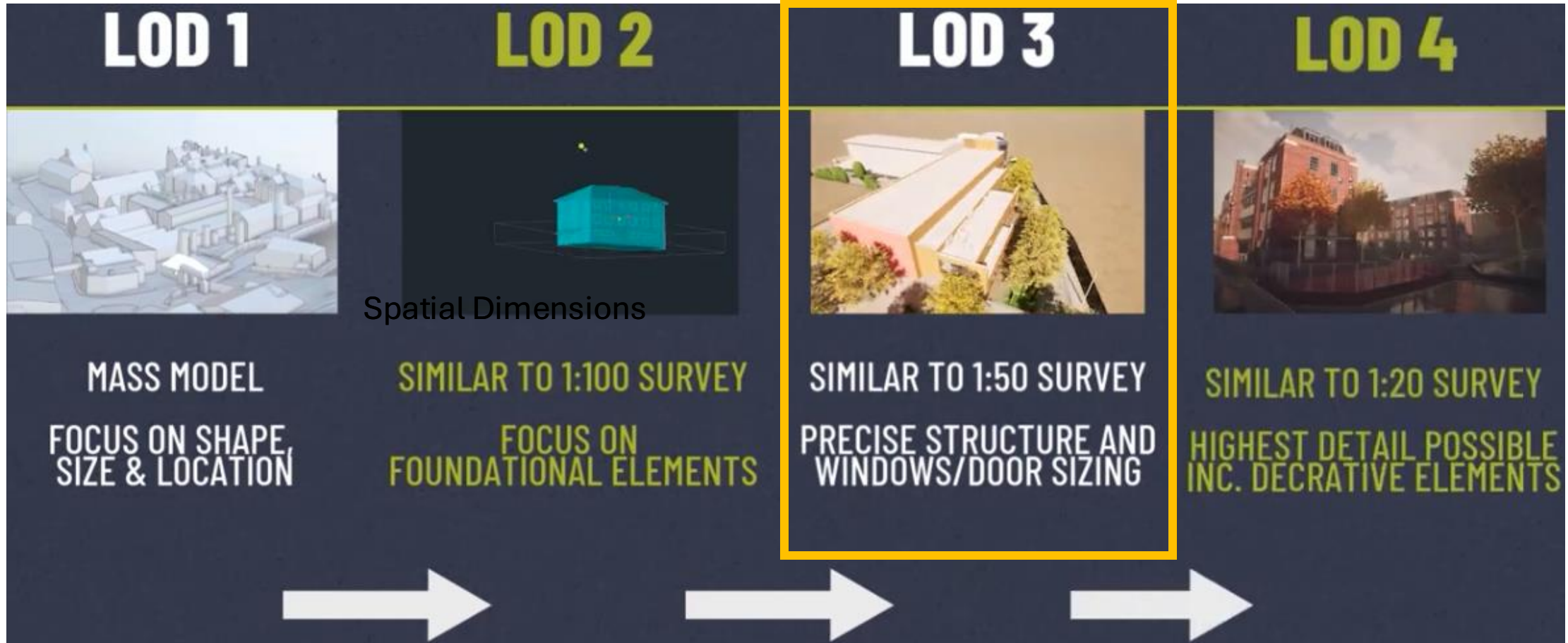
Primary focus on navigation, enabling large-scale generation from open data sources like OSM and Overture Maps



Source: Spatial Dimensions -
<https://www.youtube.com/watch?v=A9Savf24Eul>

LODs

Can include key landmarks and dominant buildings, but not optimized for streaming a large number of LOD3 models in a single view
-> LOD3 is typically unnecessary for navigation and quick orientation



Source: Spatial Dimensions -
<https://www.youtube.com/watch?v=A9Savf24Eul>

Technical Approach

- Simplifies the glTF / 3D Tiles approach by avoiding scene graphs, HLOD (with geometric error metrics), and complex adaptive index structures
- Builds on MLT encoding techniques, such as columnar layouts and lightweight encodings (see [ACM SIGSPATIAL Paper](#))
- Unifies 2D vector data and 3D models in a single format, with seamless integration into existing styling workflows
- Uses specialized data structures (tables) to efficiently encode 2D and 3D vector data
 - Extends the existing FeatureTable data structure
 - Introduces GPU-optimized tables for 3D models (GPUTables)
- Initial draft proposal by Anas Lasram: https://github.com/alasram/mlt_3d_tile_schema
- Extended draft proposal with efficient encoding strategy and additional design considerations: <https://github.com/mactrem/mlt-3d>

FeatureTable

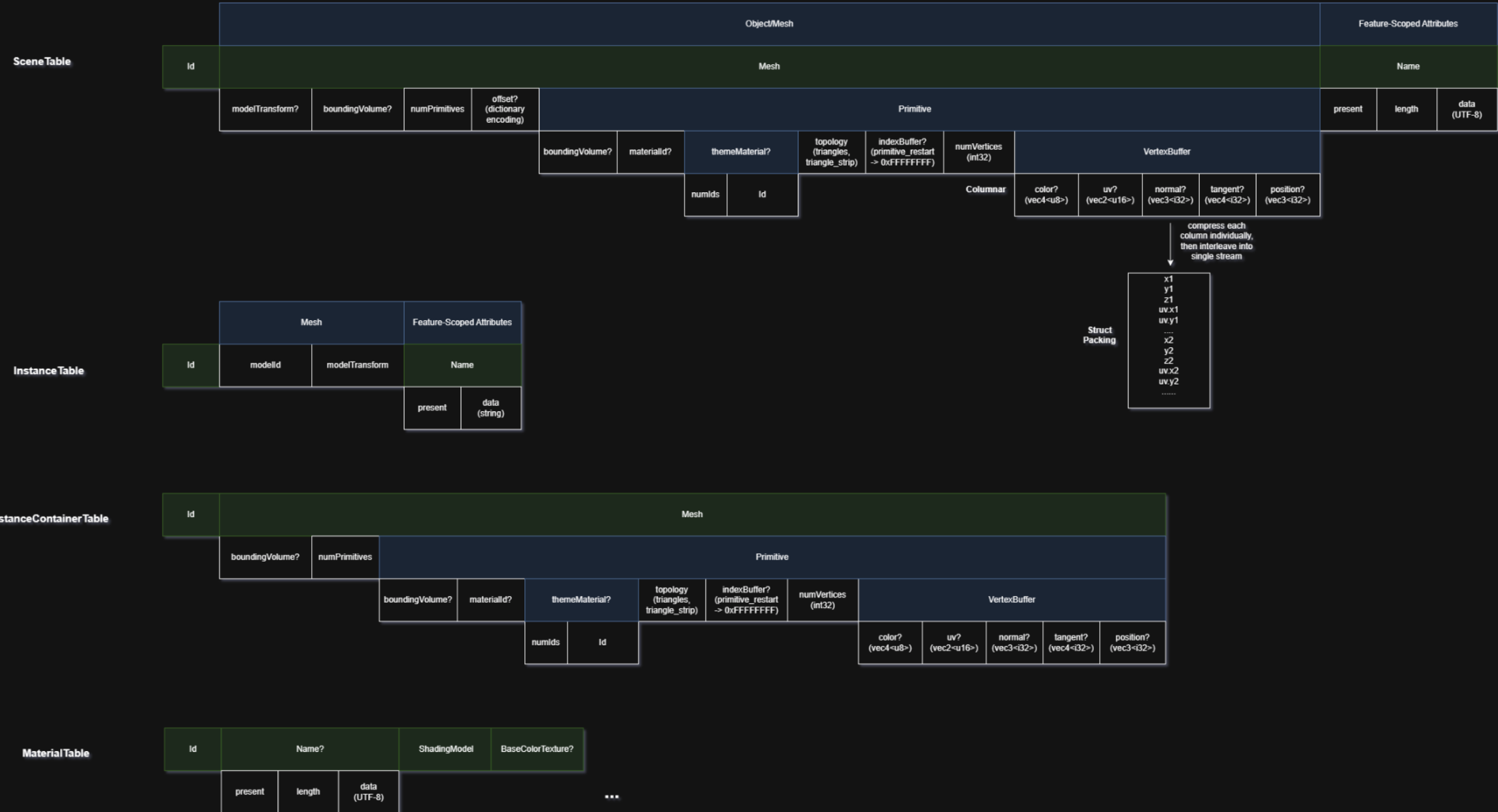
Main Idea: Favor parametric representations over pre-tessellated meshes wherever possible, leveraging procedural generation via compute or mesh shaders

Encoding Strategy: Use linear referencing to efficiently store range-based attributes

		Geometry					Feature-Scoped Attributes			Geometry-Scoped Attributes			
Id	Topology Streams		VertexBuffer Streams				Name			Width		NumLanes	
data	Type (byte)	NumVertices (int32)	UV or U? (vec2<u16> or vec<u16>?)	normal? (vec3<i32>)	tangent? (vec4<i32>)	position (vec3<i32>)	present	length	data (UTF-8)	range (vec2<i32>)	data (i32)	range (vec2<i32>)	data (u32)

GPUTables

Specialized data structures (tables) for efficient storage of 3D objects



Comparison to 3D Tiles

MLT 3D is intentionally narrower in scope than 3D Tiles/glTF. Some key differences:

- **Less generic than glTF:** No JSON-based accessor/bufferView indirection; geometry layout is defined by the table schema directly
- **Fixed quadtree tiling scheme:** MLT 3D relies on the standard XYZ/TMS quadtree pyramid, which enables random access by tile coordinate (comparable to implicit tiling in 3D Tiles), but foregoes support for adaptive and 3D spatial index structures.
- **Zoom-level LOD instead of geometric error:** LOD transitions are driven by discrete zoom levels with simple replacement refinement. There is no per-tile geometric error metric and no support for additive refinement.
- **No efficient point cloud support:** Focused on cartographic, not large-scale survey data
- **Columnar storage layout:** MLT 3D is based on a columnar storage layout with lightweight encoding schemes to achieve good compression ratio
- **Rich styling:** seamlessly integrated into the existing styling ecosystem based on the MapLibre JSON style specification